

Урок 26. Корутины. Практика. Примеры использования

27 ноября 2020

В этом уроке используем корутины в несложных примерах с репозиторием и UseCase

В уроках про Room и Retrofit мы уже примерно сформировали общее видение использования корутин в приложении: запускаем корутину и в ней вызываем suspend функции, которые делают всю фоновую работу.

В тех примерах мы вызывали suspend функции Room и Retrofit прямо из ViewModel. Но это был очень упрощенный пример, в реальном коде обычно так не делается. Между ViewModel и данными есть посредники: UseCase и репозитории.

Схематично это выглядит так:



[источник](#)

Data Source - это те самые Room DAO и Retrofit ApiService, с которыми мы работали. Они предоставляют suspend функции и Flow. На схеме видно, что функции в репозиториях и UseCase также являются suspend или возвращают Flow. А ViewModel будет использовать корутины для работы с ними.

Давайте рассмотрим пару простых сценариев, которые работают по этой схеме.

Сценарий 1

На экране отображается список пользователей. Мы можем добавлять нового пользователя или удалить сразу всех. Работаем только с локальной БД, на сервер не ходим.

Нам нужны три операции:

- получение данных
- добавление одной записи
- удаление данных

Давайте создавать объекты, которые обеспечат такой функционал. Если смотреть на картинку архитектуры выше, то мы пойдем справа налево: DAO > Repository > UseCase > ViewModel

Обращаю ваше внимание, что это не урок по архитектуре приложений. И я тут не топлю за юзкейсы. У вас все может быть совсем по другому. Цель урока - показать, как корутины могут быть встроены в архитектуру.

Data Source

DAO для работы с БД:

```
1 @Dao
2 interface UserDao {
3
4     @Query("SELECT * FROM user")
5     fun getAll(): Flow<List<UserDb>>
6
7     @Insert
8     suspend fun insert(user: UserDb)
9
10    @Query("DELETE FROM user")
11    suspend fun deleteAll()
12 }
13 }
```

Получение данных оформляем как Flow. А вставку и удаление делаем suspend функциями.

Repository

Далее - репозиторий для работы с данными пользователей.

Сразу поясню немного по архитектуре. Я здесь придерживался схемы, которая часто используется для организации типов данных. Если у нас есть тип данных User, который мы используем в UI части, то для базы данных должен быть отдельный тип UserDb. Для Retrofit, если бы он тут использовался, тоже использовался бы отдельный тип UserApi. За пределы UserRepository может выйти только тип User. Для конвертации одного типа данных в другой внутри репозитория используем маппинги.

Код:

```

1  class UserRepository(
2      private val userDao: UserDao,
3      private val userMapperDbToUi: UserMapperDbToUi,
4      private val userMapperUiToDb: UserMapperUiToDb
5  ) {
6
7      fun getUsers(): Flow<List<User>> {
8          return userDao.getAll().map { userMapperDbToUi.transformList(it) }
9      }
10
11     suspend fun addUser(user: User) {
12         userDao.insert(userMapperUiToDb.transform(user))
13     }
14
15     suspend fun deleteAllUsers() {
16         userDao.deleteAll()
17     }
18
19 }

```

Методы репозитория полностью повторяют соответствующие методы из DAO и используют маппинги для конвертации UserDb в User и наоборот.

Обратите внимание на использование suspend. Чтобы вызывать suspend функции DAO (insert и deleteAll), нам нужно использовать слово suspend и в функциях репозитория, т.к. suspend функция может быть вызвана только из корутины или другой suspend функции.

UseCase

Переходим к UseCase. Их будет три: получение пользователей, удаление пользователей, добавление пользователя.

Получение пользователей:

```

1  class GetUsersUseCase(private val userRepository: UserRepository) {
2
3      fun execute(): Flow<List<User>> {
4          return userRepository.getUsers()
5      }
6
7  }

```

Просто пробрасываем Flow из репозитория.

Удаление пользователей:

```

1  class DeleteAllUsersUseCase(private val userRepository: UserRepository) {
2
3      suspend fun execute() {
4          userRepository.deleteAllUsers()
5      }
6  }

```

Снова просто вызываем метод репозитория. А для этого нам надо объявлять метод execute как suspend.

Добавление пользователя:

```

1  class AddUserUseCase(
2      private val userRepository: UserRepository,
3      private val userValidator: UserValidator
4  ) {
5
6      suspend fun execute(user: User): Result<Unit> {
7
8          if (!userValidator.isValid(user)) {
9              return Error(IllegalArgumentException("User is not valid"))
10         }
11
12         try {
13             userRepository.addUser(user)
14         } catch (e: Exception) {
15             return Error(e)
16         }
17
18         return Success(Unit)
19     }
20 }

```

Чтобы сделать пример интереснее, я добавил немного логики в этот UseCase.

Он возвращает тип Result (Success|Error).

Перед добавлением данных пользователя в БД мы выполняем проверку, что эти данные валидны. Если данные не валидны, мы сразу возвращаем Error и ничего не вставляем в БД.

Вставка данных в БД обернута в try-catch, чтобы в случае возникновения ошибки получить объект Error.

Если все прошло хорошо, то возвращаем пустой Success.

Метод execute помечаем как suspend, потому что мы в нем вызываем suspend метод userRepository.addUser.

ViewModel

Используем созданные UseCase в ViewModel:

```
1 class MainViewModel : ViewModel(...) {
2
3     // ...
4
5     val users = getUsersUseCase.execute().asLiveData()
6
7     fun onAddClick() {
8         viewModelScope.launch {
9             val result = addUsersUseCase.execute(User(...))
10
11             if (result is Error) {
12                 showError(result.exception)
13             }
14         }
15     }
16
17
18     fun onClearClick() {
19         viewModelScope.launch {
20             deleteAllUsersUseCase.execute()
21         }
22     }
23
24     // ...
25
26 }
```

Flow из getUsersUseCase сразу конвертируем в LiveData.

По нажатию на кнопку Add запускаем корутину, а в ней вызываем suspend метод addUsersUseCase.execute. Если результат - Error, то отображаем ошибку.

По нажатию на кнопку Clear запускаем корутину, а в ней вызываем suspend метод deleteAllUsersUseCase.execute.

Т.к. мы используем Room, то добавление и удаление пользователей автоматически будут триггерить Flow из getUsersUseCase, и в LiveData придут обновленные данные.

Как видим, код ViewModel не сильно отличается от примеров из уроков про Room и Retrofit. Мы стартуем корутины и в них работаем с suspend функциями. Код репозитория и UseCase выглядит вполне обычным, с тем лишь отличием, что их функции объявлены как suspend.

Сценарий 2

Во втором сценарии рассмотрим получение данных с сервера и сохранение их в БД.

С точки зрения UI нам нужна только одна операция - получение данных. А внутри нее уже будет происходить работа с БД и сервером.

Идем по той же схеме, DAO/ApiService > Repository > UseCase > ViewModel

DataSource

ApiService для получения данных с сервера:

```
1 interface UserApiService {
2
3     @GET("users")
4     suspend fun fetchUsers(): List<UserApi>
5
6 }
```

Помечаем метод как suspend

DAO для работы с БД:

```
1 @Dao
2 interface UserDao {
3
4     @Query("SELECT * FROM user")
5     suspend fun getAll(): List<UserDb>
6
7     @Insert
8     suspend fun insertAll(users: List<UserDb>)
9
10    @Query("DELETE FROM user")
11    suspend fun deleteAll()
12
13 }
```

Три suspend метода: чтение, вставка и удаление. Обратите внимание, чтение на этот раз не Flow, а suspend. Т.е. в этом сценарии нам не нужна подписка на данные. Нужен способ получить данные в коде здесь и сейчас.

Repository

Репозиторий:

```
1 class UserRepository(  
2     private val userDao: UserDao,  
3     private val userApiService: UserApiService,  
4     private val userMapperDbToUi: UserMapperDbToUi,  
5     private val userMapperApiToDb: UserMapperApiToDb  
6 ) {  
7  
8     suspend fun getUsers(): List<User> {  
9         return userMapperDbToUi.transformList(userDao.getAll())  
10    }  
11  
12    suspend fun fetchUsers() {  
13        val usersApi = userApiService.fetchUsers()  
14        val usersDb = userMapperApiToDb.transformList(usersApi)  
15        userDao.deleteAll()  
16        userDao.insertAll(usersDb)  
17    }  
18 }  
19 }
```

Метод getUsers просто использует соответствующий метод DAO для получения данных, плюс маппинг.

А вот метод fetchUsers интереснее. Он ничего не возвращает, а только обновляет данные в БД. Для этого он получает данные с сервера, конвертирует их и вставляет в БД, предварительно удалив все старые данные.

Т.е. в одном suspend методе мы вызываем три suspend метода.

UseCase

Переходим к UseCase:

```
1 class FetchOrGetUsersUseCase(  
2     private val userRepository: UserRepository,  
3     private val networkHelper: NetworkHelper,  
4     private val preferences: Preferences  
5 ) {  
6  
7     suspend fun execute(): List<User> {  
8         if (networkHelper.isWiFi()) {  
9             try {  
10                userRepository.fetchUsers()  
11            } catch (e: Exception) {  
12                // ...  
13            }  
14            preferences.lastUpdated()  
15        }  
16        return userRepository.getUsers()  
17    }  
18 }  
19 }
```

Сначала проверяем, что мы подключены к WiFi. В этом случае вызываем метод fetchUsers, который загрузит данные с сервера и запишет их в БД. Т.к. при этом мы совершаем запрос к серверу, то вполне могут возникнуть ошибки. Поэтому вызов оборачиваем в try-catch.

Если обновление данных прошло успешно, то методом lastUpdated фиксируем время обновления данных. Это нужно, например, если мы хотим где-нибудь отображать, насколько актуальны данные в БД.

В конце мы читаем данные из БД методом getUsers и возвращаем их. Если не было WiFi, то эти данные просто будут не самыми свежими.

ViewModel

Смотрим на код ViewModel:

```
1 class MainViewModel : ViewModel() {  
2  
3     val users = MutableLiveData<List<User>>()  
4  
5     fun getData() {  
6         viewModelScope.launch {  
7             showLoading()  
8             val usersData = fetchOrGetUsersUseCase.execute()  
9             hideLoading()  
10            users.value = usersData  
11        }  
12    }  
13 }  
14 }
```

Запускаем корутину. В ней показываем индикатор загрузки, используем UseCase, чтобы получить данные, скрываем индикатор загрузки, помещаем данные в LiveData.

Код ViewModel снова не особо сложный. В корутине вызываем suspend метод, внутри которого крутится вся логика и вызывается несколько других suspend методов.

Этот сценарий вполне можно было сделать по-другому: разбить FetchOrGetUsersUseCase на два разных UseCase. Первый читает данные из БД с помощью Flow. А второй отвечает только за получение данных с сервера и обновление их в БД, и ничего не возвращает. В связке они будут отлично работать.

Давайте отдельно обсудим некоторые вопросы, которые могут возникнуть при написании такого кода.

suspend функции

В примерах выше для работы с БД и сетью мы использовали Room и Retrofit. Они поддерживают Flow и suspend функции из коробки.

Если же у нас есть просто долгий синхронный код или код с колбэком, то мы можем сами создать для них suspend обертку. В случае синхронных функций можно просто использовать withContext, как мы это делали в Уроках 24 и 25. А для колбэков используйте suspendCoroutine или suspendCancellableCoroutine, которые мы разбирали в Уроках 3 и 16.

Обработка исключений

Чтобы поймать исключение, пришедшее из suspend, мы просто оборачиваем вызов suspend функции в try-catch. Корутина при этом отменена не будет.

Отмена корутины

Как мы знаем, если закрыть экран, то ViewModel будет уничтожена. А следовательно будет отменен viewModelScope и все его корутины. Как поведет себя suspend функция в этом случае? Выбросит исключение CancellationException (если она cancellable, Урок 16).

Flow прекратит свою работу, если его collect был вызван в корутине, которую мы отменяем (Урок 22).

И не забывайте, что asLiveData под капотом использует билдер liveData, который НЕ использует viewModelScope. Вместо этого он создает свой Scope, который привязан к Lifecycle подписчика его LiveData. Если перевести на русский, то Flow, к которому применили asLiveData, будет работать, пока есть подписчик на его LiveData.

Защита от повторных вызовов

Вернемся к примеру получения данных с сервера:

```
1 class MainViewModel : ViewModel() {
2
3     val users = MutableLiveData<List<User>>()
4
5     fun getData() {
6         viewModelScope.launch {
7             showLoading()
8             val usersData = fetchOrGetUsersUseCase.execute()
9             hideLoading()
10            users.value = usersData
11        }
12    }
13
14 }
```

Если метод getData вызывается по нажатию на кнопку, и пользователь будет жать эту кнопку несколько раз, то мы получим множественные вызовы. Каждый такой вызов создает новую корутину. Каждая корутина запрашивает данные.

В итоге будет куча лишних одновременных вызовов. И индикатор загрузки будет моргать. Давайте это исправим.

Перед тем, как вызывать новую корутину, будем проверять, что прошлая уже завершилась. В этом нам поможет Job:

```
1 var getDataJob: Job? = null
2
3 fun getData() {
4     if (getDataJob?.isActive == true) return
5
6     getDataJob = viewModelScope.launch {
7         showLoading()
8         val usersData = fetchOrGetUsersUseCase.execute()
9         hideLoading()
10        users.value = usersData
11    }
12 }
```

Схема очень похожа на работу с Disposable в RxJava. Когда мы запускаем корутину, мы сохраняем ее Job. При следующем вызове getData мы выполняем проверку. Если Job еще работает, значит прошлая корутина все еще получает данные. Ничего не делаем. А если Job уже завершился, значит прошлая корутина уже отработала и можно запросить данные снова.